



25 years of the free encyclopedia

Linear search

In computer science, **linear search** or **sequential search** is a method for finding an element within a list. It sequentially checks each element of the list until a match is found or the whole list has been searched.^[1]

A linear search runs in linear time in the worst case, and makes at most n comparisons, where n is the length of the list. If each element is equally likely to be searched, then linear search has an average case of $\frac{n+1}{2}$ comparisons, but the average case can be

affected if the search probabilities for each element

vary. Linear search is rarely practical because other search algorithms and schemes, such as the binary search algorithm and hash tables, allow significantly faster searching for all but short lists.^[2]

Linear search

<u>Class</u>	<u>Search algorithm</u>
<u>Worst-case performance</u>	<u>$O(n)$</u>
<u>Best-case performance</u>	<u>$O(1)$</u>
<u>Average performance</u>	<u>$O(n)$</u>
<u>Worst-case space complexity</u>	<u>$O(1)$</u> iterative
<u>Optimal</u>	Yes

Algorithm

A linear search sequentially checks each element of the list until it finds an element that matches the target value. If the algorithm reaches the end of the list, the search terminates unsuccessfully.^[1]

Basic algorithm

Given a list L of n elements with values or records $L_0 \dots L_{n-1}$, and target value T , the following subroutine uses linear search to find the index of the target T in L .^[3]

1. Set i to 0.
2. If $L_i = T$, the search terminates successfully; return i .
3. Increase i by 1.
4. If $i < n$, go to step 2. Otherwise, the search terminates unsuccessfully.

We can define this in pseudocode as given below, using either an iterative or recursive approach.

```
function iterativeLinearSearch(list L, T) is
    for i = 0 to length(L) do
        if L[i] == T then
            return i
    // Return an unsuccessful value (in this case -1).
    return -1
```

```
function recursiveLinearSearch(list L, T, i = 0) is
    if L[i] == T then
        return i
    if i > length(L) then
        return -1 // Unsuccessful value.
    return recursiveLinearSearch(L, T, i = i + 1)
```

With a sentinel

The basic algorithm above makes two comparisons per iteration: one to check if L_i equals T , and the other to check if i still points to a valid index of the list. By adding an extra record L_n to the list (a sentinel value) that equals the target, the second comparison can be eliminated until the end of the search, making the algorithm faster. The search will reach the sentinel if the target is not contained within the list.^[4]

1. Set i to 0.
2. If $L_i = T$, go to step 4.
3. Increase i by 1 and go to step 2.
4. If $i < n$, the search terminates successfully; return i . Else, the search terminates unsuccessfully. We can define this in pseudocode as given below, using either an iterative or recursive approach.

```
function iterativeSentinelSearch(list L, T) is
    for i = 0 to length(L) do
        if L[i] == T then
            if i < length(L) then
                return i
            else
                return -1
    return -1

function recursiveSentinelSearch(list L, T, i = 0) is
    if i >= length(L) then
        return -1
    if L[i] == T then
        return i
    return recursiveSentinelSearch(L, T, i = i + 1)
```

In an ordered table

If the list is ordered such that $L_0 \leq L_1 \dots \leq L_{n-1}$, the search can establish the absence of the target more quickly by concluding the search once L_i exceeds the target. This variation requires a sentinel that is greater than the target.^[5]

1. Set i to 0.
2. If $L_i \geq T$, go to step 4.
3. Increase i by 1 and go to step 2.
4. If $L_i = T$, the search terminates successfully; return i . Else, the search terminates unsuccessfully. We can define this in pseudocode as given below, using either an iterative or

recursive approach.

```
function iterativeTableSearch(list L, T) is
  for i = 0 to length(L) do
    if L[i] >= T then
      if L[i] == T then
        return i
      else
        return -1
  return -1

function recursiveTableSearch(list L, T, i = 0) is
  if i >= length(L) then
    return -1
  if L[i] >= T then
    if L[i] == T then
      return i
    else
      return -1
  return recursiveTableSearch(L, T, i = i + 1)
```

Analysis

For a list with n items, the best case is when the value is equal to the first element of the list, in which case only one comparison is needed. The worst case is when the value is not in the list (or occurs only once at the end of the list), in which case n comparisons are needed.

If the value being sought occurs k times in the list, and all orderings of the list are equally likely, the expected number of comparisons is

$$\begin{cases} n & \text{if } k = 0 \\ \frac{n+1}{k+1} & \text{if } 1 \leq k \leq n. \end{cases}$$

For example, if the value being sought occurs once in the list, and all orderings of the list are equally likely, the expected number of comparisons is $\frac{n+1}{2}$. However, if it is *known* that it occurs once, then at most $n - 1$ comparisons are needed, and the expected number of comparisons is

$$\frac{(n+2)(n-1)}{2n}$$

(for example, for $n = 2$ this is 1, corresponding to a single if-then-else construct).

Either way, asymptotically the worst-case cost and the expected cost of linear search are both $O(n)$.

Non-uniform probabilities

The performance of linear search improves if the desired value is more likely to be near the

beginning of the list than to its end. Therefore, if some values are much more likely to be searched than others, it is desirable to place them at the beginning of the list.

In particular, when the list items are arranged in order of decreasing probability, and these probabilities are geometrically distributed, the cost of linear search is only $O(1)$.^[6]

In general, if items are arranged in order of decreasing probability and the probability of searching for the i th element is p_i , the expected cost of a single search is $ES^{OPT} = \sum_{i=1}^n ip_i$.

Under the natural assumption that the probabilities are not known in advance, or one cannot spend the time to sort the list by probabilities, one can use the approach of *self-adjusting data structure* and move elements towards the head of the list when they are requested in a search. Two natural heuristics for this self-adjustment are *Move to Front* (MF) and *Transpose* (T), where the requested element trades places with its predecessor. It is known that the expected cost of an access in a large sequence of independent accesses, averaged over all initial orders of the list, satisfies $ES^T \leq ES^{MF} \leq \frac{\pi}{2} ES^{OPT}$. In terms of amortized cost, averaging over a worst-case sequence of operations (note - among sequences satisfying the assumption on probabilities), we have $S^{MF} \leq 2S^{OPT}$, while S^T can be as bad as $O(mS^{OPT})$.^[7]

Application

Linear search is usually very simple to implement, and is practical when the list has only a few elements, or when performing a single search in an unordered list.

When many values have to be searched in the same list, it often pays to preprocess the list in order to use a faster method. For example, one may sort the list and use binary search, or build an efficient search data structure from it. Should the content of the list change frequently, repeated reorganization may be more trouble than it is worth.

As a result, even though in theory other search algorithms may be faster than linear search (for instance binary search), in practice even on medium-sized arrays (around 100 items or less) it might be infeasible to use anything else. On larger arrays, it only makes sense to use other, faster search methods if the data is large enough, because the initial time to prepare (sort) the data is comparable to many linear searches.^[8]

See also

- Ternary search
- Hash table
- Linear search problem

References

Citations

1. Knuth 1998, §6.1 ("Sequential search").
2. Knuth 1998, §6.2 ("Searching by Comparison Of Keys").
3. Knuth 1998, §6.1 ("Sequential search"), subsection "Algorithm B".
4. Knuth 1998, §6.1 ("Sequential search"), subsection "Algorithm Q".
5. Knuth 1998, §6.1 ("Sequential search"), subsection "Algorithm T".
6. Knuth, Donald (1997). "Section 6.1: Sequential Searching". *Sorting and Searching*. The Art of Computer Programming. Vol. 3 (3rd ed.). Addison-Wesley. pp. 396–408. ISBN 0-201-89685-0.
7. Baeza-Yates, Ricardo; Poblete, Patricio V. (1999). "Chapter 2: Searching". In Atallah (ed.). *Algorithms and Theory of Computation Handbook*. CRC Press. pp. 2–3. ISBN 0849326494.
8. Horvath, Adam. "Binary search and linear search performance on the .NET and Mono platform" (<http://blog.teamleadnet.com/2012/02/quicksort-binary-search-and-linear.html>). Retrieved 19 April 2013.

Works

- Knuth, Donald (1998). *Sorting and Searching*. The Art of Computer Programming. Vol. 3 (2nd ed.). Reading, MA: Addison-Wesley Professional. ISBN 0-201-89685-0

Retrieved from "https://en.wikipedia.org/w/index.php?title=Linear_search&oldid=1358295419"